دانشگاه فردوسی مشهد

دانشکده مهندسی

گروه آموزشی مهندسی کامپیوتر

پیاده‌سازی wordcount با Map/Reduce در Go

موعد تحویل	۸ آبان ۱۴۰۵

هدف تمرین

هدف این تمرین، پیاده‌سازی الگوی برنامه‌نویسی Map/Reduce به صورت شبیه‌سازی‌شده بر روی یک ماشین واحد است. شما با استفاده از زبان برنامه‌نویسی Go و قابلیت‌های همزمانی آن (goroutines و channels)، یک برنامه شمارش کلمات (Word Count) را برای مجموعه‌ای از فایل‌های ورودی پیاده‌سازی خواهید کرد. در این تمرین هر مرحله از Map/Reduce توسط چند goroutine مستقل انجام می‌شود که فقط از طریق channel با هم ارتباط دارند. این ساختار شبیه سیستم‌های توزیع‌شده‌ی واقعی است، با این تفاوت که به‌جای چند ماشین، چند goroutine داریم. این تمرین به شما کمک می‌کند با مفاهیم اصلی Map/Reduce یعنی تقسیم مسئله (Map)، تجمیع نتایج میانی (Shuffle/Group) و ترکیب نهایی (Reduce) به صورت عملی آشنا شوید.

صورت مسئله

برنامه‌ای به زبان Go بنویسید که:

1. نام پنج فایل متنی (`input1.txt` تا `input5.txt`) را به عنوان ورودی دریافت کند. اگر آرگومانی از خط فرمان داده نشد، از همین پنج نام به عنوان پیش‌فرض استفاده کند.
2. محتوای این فایل‌ها را با استفاده از goroutineهای موازی پردازش کند.
3. برای هر فایل، یک goroutine اجرا شود که تابع `Map` را صدا بزند. این تابع کلمات موجود در فایل را استخراج کرده و به صورت جفت‌های کلید-مقدار (`"word", "1"`) تولید می‌کند. این جفت‌ها باید از طریق یک channel به مرحله‌ی بعد ارسال شوند.

4. نتایج میانی تولیدشده توسط تمام توابع `Map` را تجمیع (Shuffle/Group) کند. به این معنی که تمام مقادیر مربوط به یک کلید (کلمه) خاص، در یک لیست کنار هم قرار گیرند. برای مثال:

```
"hello" → ["1", "1", "1"]
```

کلیدها باید با رابطه‌ی `hash(key) % nReduce` بین `nReduce` پارتیشن تقسیم شوند (مقدار `nReduce` باید در برنامه قابل تنظیم باشد). هر کلید همیشه باید به یک پارتیشن مشخص برود.

5. برای هر پارتیشن، یک goroutine کاهنده (Reduce) اجرا شود. این goroutine برای هر کلید موجود در پارتیشن خودش تابع `Reduce` را صدا می‌زند. تابع `Reduce` لیست مقادیر را گرفته و نتیجه نهایی را محاسبه می‌کند (در این مثال، جمع مقادیر). نتیجه‌ی هر کلید باید روی یک channel ارسال شود.

6. خروجی نهایی، یعنی تعداد کل تکرار هر کلمه در تمام پنج فایل، در یک فایل متنی به نام `output.txt` و به صورت مرتب‌شده بر اساس کلمه، ذخیره شود. هر خط از فایل خروجی باید به فرمت `word: count` باشد.

الزامات پیاده‌سازی

- کلمات باید بدون حساسیت به حروف بزرگ و کوچک شمرده شوند (The و the یک کلمه‌اند) و علائم نگارشی جزو کلمه نیستند.
- برای هماهنگی بین goroutineها از `sync.WaitGroup` و channel استفاده کنید. دسترسی همزمان چند goroutine به یک متغیر مشترک بدون محافظت مجاز نیست.
- اگر یکی از فایل‌های ورودی وجود نداشت، برنامه نباید crash کند. باید پیام خطا چاپ شود و بقیه‌ی فایل‌ها پردازش شوند.
- برنامه باید با دستور `go run -race main.go` بدون هشدار اجرا شود.

نمونه

اگر `input1.txt` شامل `the quick brown fox jumps over the lazy dog` و `input2.txt` شامل `the dog barks and the fox runs away` باشد، بخشی از خروجی مورد انتظار برای این دو فایل به صورت زیر است:

```
and: 1
away: 1
barks: 1
brown: 1
dog: 2
fox: 2
the: 4
```

موارد مورد نیاز برای تحویل

1. سورس کد کامل برنامه.
2. فایل خروجی `output.txt` که بر اساس فایل‌های ورودی ارائه شده.
3. مستندات (داکیومنت) پروژه که شامل توضیح ساختار برنامه، نحوه‌ی پیاده‌سازی هر مرحله و روش اجرای آن باشد.